

Dynamic

Video-131

Programming



Note :- This playlist is only for explanation of Dns & solutions.
See my "DP Concepts & Dns"
Playlist for understanding
DP from scratch...



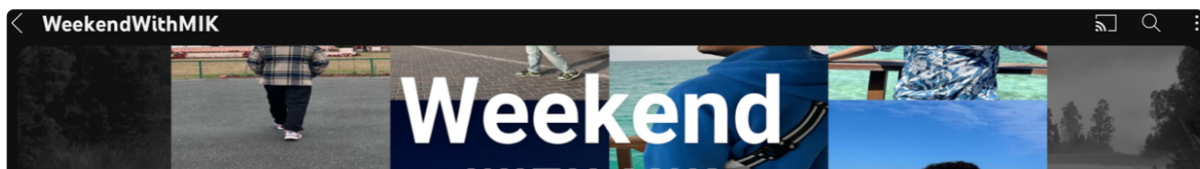
codestorywithmik



CSwithMIK



codestorywithMIK





Motivation :-

Whatever happens,
just Don't Quit.



MIK...

3651. Minimum Cost Path with Teleportations

Hard

Topics

Companies

Hint

You are given a $m \times n$ 2D integer array `grid` and an integer k . You start at the top-left cell $(0, 0)$ and your goal is to reach the bottom-right cell $(m - 1, n - 1)$.

There are two types of moves available:

- **Normal move**: You can move right or down from your current cell (i, j) , i.e. you can move to $(i, j + 1)$ (right) or $(i + 1, j)$ (down). The cost is the value of the destination cell.
- **Teleportation**: You can teleport from any cell (i, j) , to any cell (x, y) such that $grid[x][y] \leq grid[i][j]$; the cost of this move is 0. You may teleport at most k times.

Return the minimum total cost to reach cell $(m - 1, n - 1)$ from $(0, 0)$.

Example:- $grid =$

	0	1	2
0	1	3	3

, $k = 2$

1	2	5	4
2	4	3	5

$$2 + 5 = 7$$

using
1 step

Output : 7

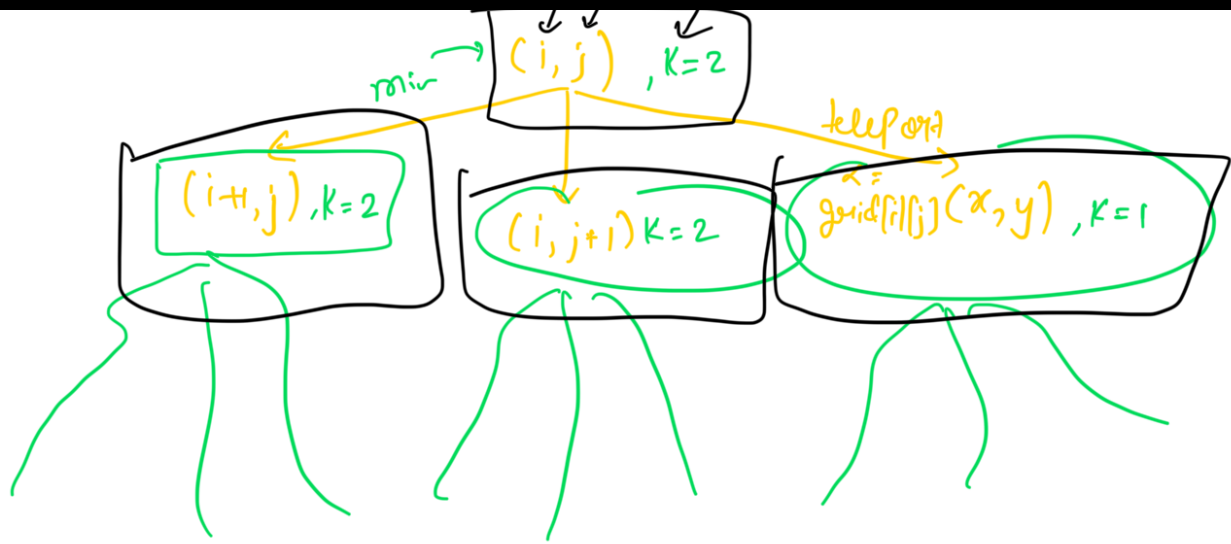
Constraints:

- $2 \leq m, n \leq 80$
- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $0 \leq \text{grid}[i][j] \leq 10^4$
- $0 \leq k \leq 10$

Thought Process

	0	1	2
0	1	3	3
1	2	5	4
2	4	3	5

K = 2



$(m-1)(n-1)$

Memoize: $\begin{bmatrix} & & \\ & & \\ & & \end{bmatrix}$

3D-DP

Solve $(0, 0, 0, K, \text{grid})$;

Solve $(i, j, t, K, \text{grid})$ {
 if $(i == m-1 \ \& \ j == n-1)$ {
 return 0;
 }
 }

int result = 1e9;

// Right $(i, j) \rightarrow (i, j+1)$

$\text{grid}[i][j+1];$

if $(j+1 < n)$ {

int next = Solve $(i, j+1, t, K, \text{grid});$

result = min(result, $\text{grid}[i][j+1] + \text{next}$);

}

// $(i, j) \rightarrow (i+1, j)$

if $(i+1 < m)$ {

$f(i, j, t)$

```
int next = Solve(i+1, j, t+1, K, grid);  
result = min(result, grid[i+1][j] + next);  
}
```

// helper (i, j) $\rightarrow$ (x, y)

```
if (t < K) {
```

```
    curval = grid[i][j];
```

```
    for (int x = 0; x < m; x++)
```

```
        for (int y = 0; y < n; y++) {
```

```
            if (x == i && y == j)  
                continue;
```

```
            if (grid[x][y] <= grid[i][j]) {
```

```
                result = min(result,
```

```
                Solve(x, y, t+1, K, grid);  
            }  
        }
```

```
    }
```

```
return result;
```

```
}
```

$[m][n][K]$

$$T.C = m \cdot n \cdot K * (m * n)$$

$$= O(m^2 \cdot n^2 \cdot K) \quad \underline{T.C.E.}$$

Bottom up :-

```
int solve(int i, int j, int tPort, int k, vector<vector<int>>& grid) {
    // Reached destination
    if (i == m - 1 && j == n - 1)
        return 0;

    if (t[i][j][tPort] != -1)
        return t[i][j][tPort];

    int result = 1e9; // Large value
    int curVal = grid[i][j];

    // Move Right
    if (j + 1 < n) {
        int next = solve(i, j + 1, tPort, k, grid);
        result = min(result, grid[i][j + 1] + next);
    }

    // Move Down
    if (i + 1 < m) {
        int next = solve(i + 1, j, tPort, k, grid);
        result = min(result, grid[i + 1][j] + next);
    }

    // Teleport
    if (tPort < k) {
        for (int x = 0; x < m; x++) {
            for (int y = 0; y < n; y++) {
                if ((x != i || y != j) && grid[x][y] <= curVal) {
                    result = min(result, solve(x, y, tPort + 1, k, grid));
                }
            }
        }
    }

    return t[i][j][tPort] = result;
}
```

$$dp = [m+1][n+1][k+1]$$

$dp[i][j][tPort] = \text{min cost to reach } (m-1, n-1) \text{ from } (i, j) \text{ having used } tPort$

$$dp[i][j][tPort] =$$

$$dp[x][y][tPort+1]$$

①

```
for (t = 0; t <= K; t++) {
    dp[m-1][n-1][t] = 0;
}
```

②





```
for (tpart = K; tpart >= 0; tpart--) {
```

```
    for (int i = m-1; i >= 0; i--) {
```

```
        for (int j = n-1; j >= 0; j--) {
```

```
            if (i == m-1 & j == n-1) {
                continue;
```

```
            }
```

```
            int result = 1e9;
```

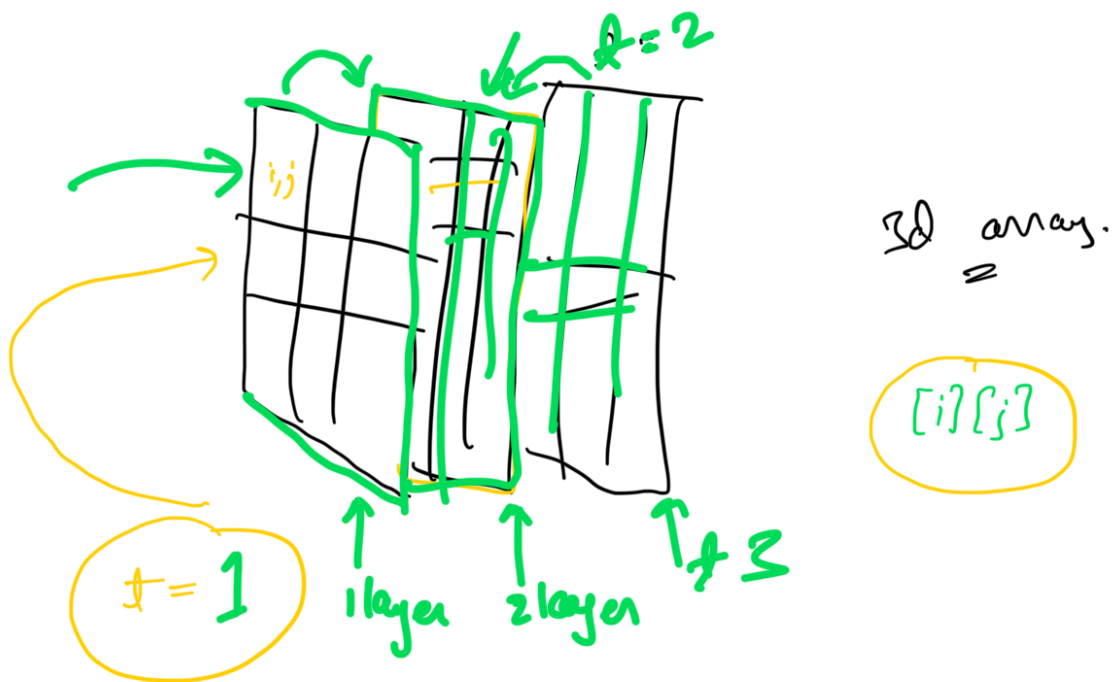
```
            curr = grid[i][j];
```

T.L.E.

```
        }
```

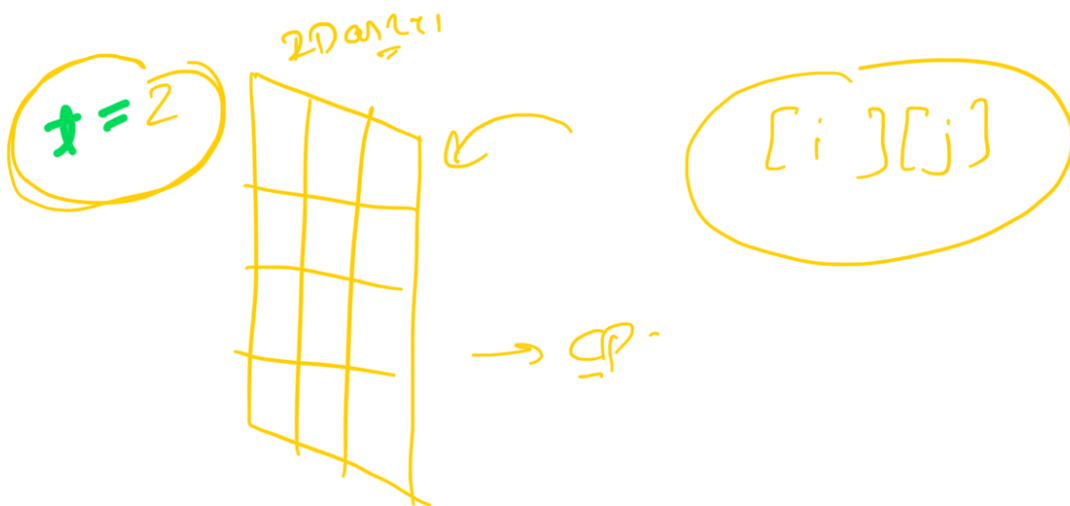
```
    return dp[0][0][0];
```

How to further optimize it ???

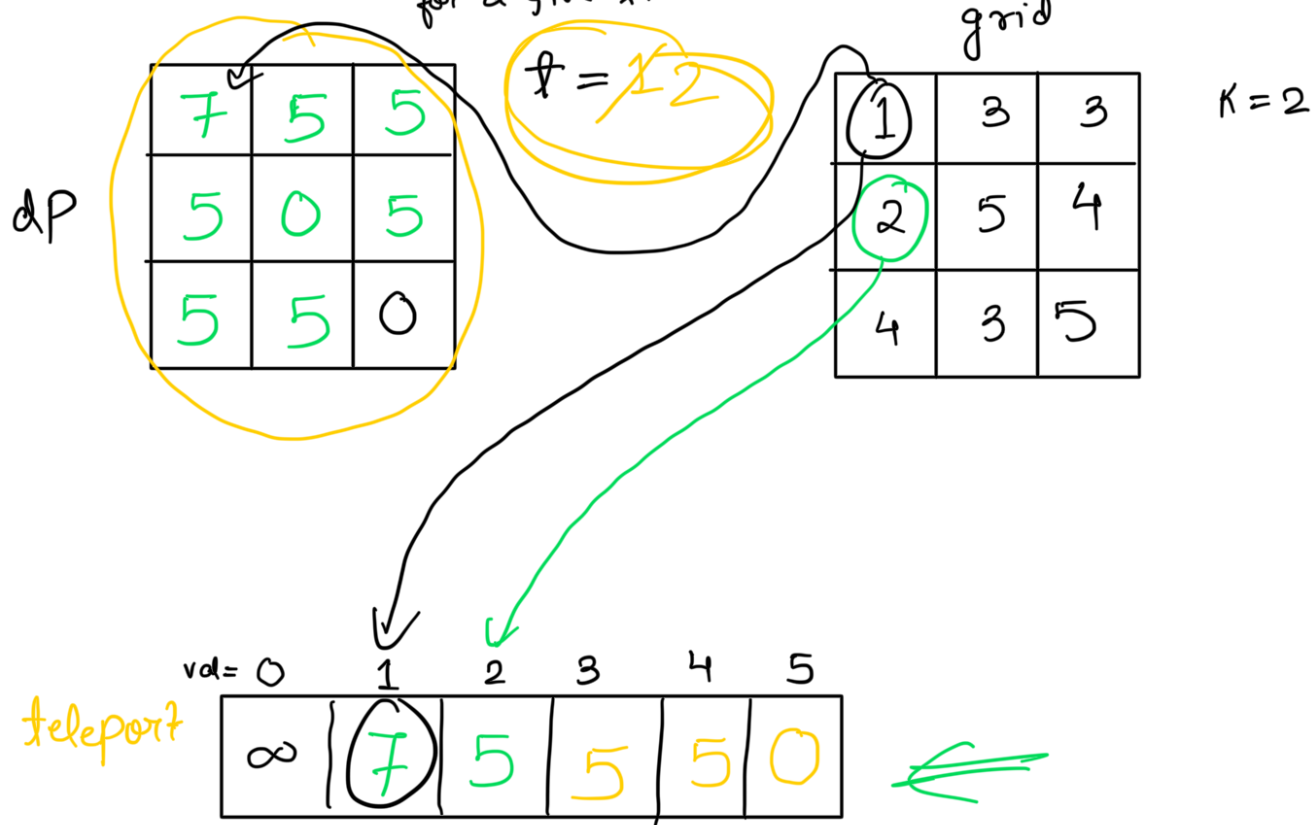


"Layered D.P"

It's just a DP technique in which we solve the same problem multiple times, each time allowing one more unit of a limited resource (step -)



$dp[i][j] = \min(\text{cost}(i,j) \rightarrow (m,n-1))$
for a give t .



teleport(val) =



for ($t=0$; $t \leq k$; $t++$) $\rightarrow O(k)$

```

for (i = m-1; i >= 0; i--)  $\rightarrow O(m)$ 
{
    for (j = n-1; j >= 0; j--)  $O(n)$ 
    {
        //
        if (i+1 < m) {
             $dp[i][j] = \min(dp[i][j], dp[i+1][j] + \text{grid}[i+1][j]);$ 
        }

        if (j+1 < n) {
             $dp[i][j] = \min(dp[i][j], dp[i][j+1] + \text{grid}[i][j+1]);$ 
        }

        if (t > 0) {
             $dp[i][j] = \min(dp[i][j], \text{teleport} + \text{grid}[i][j]);$ 
        }
    }
}

```

```

for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
         $dp[i][j] = \min(\text{teleport} + \text{grid}[i][j], dp[i][j]);$ 
    }
}

```

```
// Prefix-min → Correctly update the dp
for(int i=1; i<telep.size(); i++) {
    telep[i] = min(telep[i], telep[i-1]);
}
```

return telep[0][0];

$$T.C = O(K \cdot (m \cdot n + m \cdot n))$$

$$T.C = \underline{\underline{O(K \cdot m \cdot n)}}.$$

What went
wrong?

dp

dp

5	4
4	0

t = 1

grid

3	1
10	4

K=2
0+4=4

0

